

CS 498: Machine Perception

HW1: Registration, Odometry, and Semantic Mapping

Fall 2026

Course weight	15%
Work mode	Individual
Release	Friday, September 18, 2026
Due	Wednesday, October 7, 2026 at 11:59 p.m. America/Chicago

1 What you will build

An autonomous vehicle receives one LiDAR scan at a time. Each scan starts in a different sensor frame, individual correspondences can be wrong, and small motion errors accumulate as the vehicle moves. To build a useful spatial memory, we must align scans, estimate a trajectory, and combine repeated semantic observations in a common world frame.

You will first implement rigid fitting and RANSAC. You will then debug your own ICP on two real 3D fragments and run the same implementation across a long 1,024-frame LiDAR sequence. Finally, you will aggregate all 1,024 frames into a sparse semantic occupancy map and render each occupied voxel as a 3D cube.

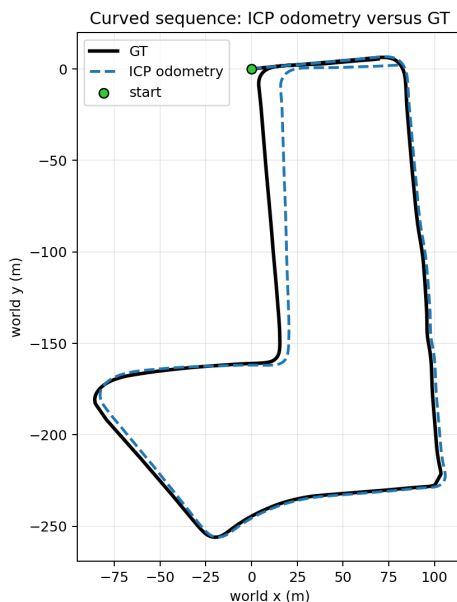


Figure 1: The curved odometry result compares only the handwritten ICP trajectory with the ground-truth trajectory, using equal world-axis scale.

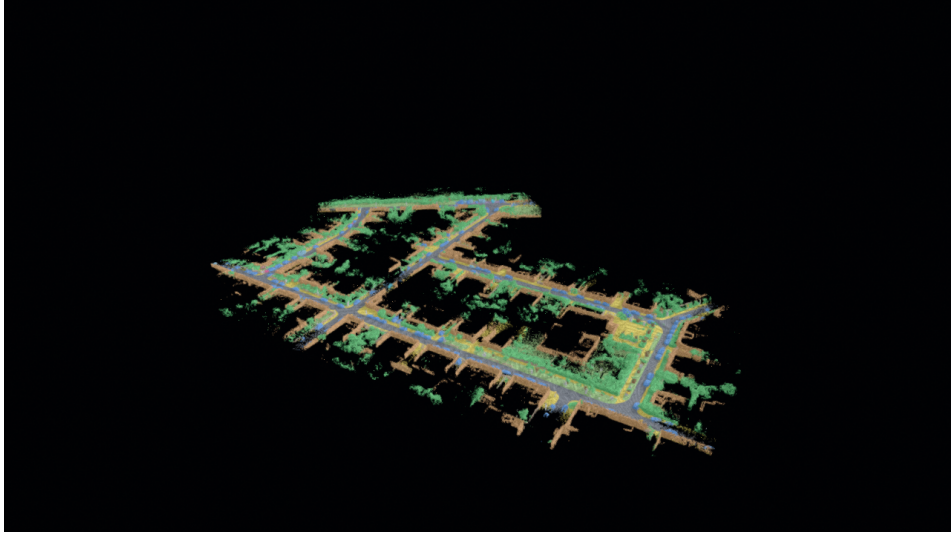


Figure 2: The final semantic deliverable: all 1,024 frames fused into equal-metric Open3D cube primitives and viewed from a shoulder camera. You will also produce a turntable and a moving LiDAR/map comparison.

2 Coordinates, transforms, and supplied data

We follow the lecture convention: $\mathbf{T}_{A \rightarrow B}$ (abbreviated $\mathbf{T}_{AB}$ in the slides) maps coordinates from frame A into frame B :

$$\begin{bmatrix} \mathbf{p}_B \\ 1 \end{bmatrix} = \mathbf{T}_{A \rightarrow B} \begin{bmatrix} \mathbf{p}_A \\ 1 \end{bmatrix}, \quad \mathbf{T}_{A \rightarrow B} = \begin{bmatrix} \mathbf{R}_{A \rightarrow B} & \mathbf{t}_{A \rightarrow B} \\ 0 & 1 \end{bmatrix}.$$

Code stacks N homogeneous point columns into $\tilde{\mathbf{P}}_A \in \mathbb{R}^{4 \times N}$, so the same transform acts on the entire point set directly:

$$\tilde{\mathbf{P}}_B = \mathbf{T}_{A \rightarrow B} \tilde{\mathbf{P}}_A.$$

In code, this is simply `points_B=T_A_to_B@points_A`. Raw library arrays are converted from $N \times 3$ to $4 \times N$ once when loaded; conversion back to $N \times 3$ happens only at SciPy/Open3D plotting boundaries. Transform composition uses the same matrix multiplication, with the rightmost transform acting first. For example, $\mathbf{T}_{A \rightarrow C} = \mathbf{T}_{B \rightarrow C} \mathbf{T}_{A \rightarrow B}$. Frame L_0 is the world frame W . Velodyne coordinates are x forward, y left, and z up. Plot the 2D trajectory using world x and y . The archive entry `poses_lidar_to_world[i]` is $\mathbf{T}_{L_i \rightarrow W}$.

Run commands from the directory containing the task scripts (`hw1/` in the student repository). Place these required files in its `data/` subdirectory:

- **registration_demo.npz**: two real fragments from Open3D’s ICP demo, 1,000 real corresponding pairs, and a deliberately perturbed initial transform for ICP debugging;
- **sequence00.npz**: SemanticKITTI sequence-00 frames 2336–3359, totaling 1,024 frames and 8,192,000 spatially sampled points.

The selected LiDAR path travels about 870 m and includes a large loop with a clear revisit. Each frame contains at most 8,000 points after deterministic 0.25 m voxel sampling. Frame boundaries are stored in `frame_offsets`. The archive also supplies reference poses, semantic labels, and 19-class logits derived from those labels. A valid point has score +4 for its GT class and −4 for the others; ignore points contain zeros. Thus Task 4 tests mapping and evidence fusion, not a learned semantic network. See `data/README.md` for exact keys, dtypes, hashes, and provenance.

3 Run one file per task

Use Python 3.12 in a virtual environment. Download both archives from https://drive.google.com/drive/folders/13azu-aNzMn3jrZ_soxHsX5SN1RcGdr64?usp=sharing. The README gives resumable gdown commands. Place the two NPZ files directly in `data/`, then run:

```
python -m pip install -r requirements.txt
python preflight.py --render
python task1_alignment.py
python task2_ransac.py
python task3_odometry.py
python task4_mapping.py
```

The preflight verifies archive hashes, then tests graphics and MP4 encoding in an isolated process. If graphics fail, use `--skip-rendering` on any task to continue numerical work and 2D diagnostics. This skips Open3D outputs, the online video, and the PLY. Contact staff with preflight output to arrange rendering on a supported machine; all report outputs remain required. Do not modify helpers to repair graphics. On Linux, EGL/Mesa or GPU drivers are required in addition to Python packages. Expect several minutes and several GB of RAM for full runs; the previous staff mapping run used about 4 GB RAM. Task 3 prints progress every 50 pairs. Complete Task 1 before Tasks 2 and 3; Task 4 uses supplied GT poses independently. Run `python-munittestdiscover-stests` for small numerical examples. Tests are expected to fail before implementation and do not replace full runs. The supplied loader projects the rounded demo rotations onto $SO(3)$ before use and evaluation; the frozen archive bytes are unchanged.

Edit only code between “Task N: Modify your code below” and “DO NOT MODIFY CODE OUTSIDE THE BLOCK.” Every file has a visible `main()` and saves a checkpoint after each subtask. Untouched starters run to labeled placeholder outputs without raising an exception. Do not modify `hw1_utils.py`; it only contains provided rendering and output machinery, and you do not need to read its internals. The supplied code uses Matplotlib only for genuine 2D diagnostics (residual histograms, convergence, trajectories, and BEV). Every 3D scene is rendered in Open3D with unchanged metric coordinates.

File	Functions you implement
<code>task1_alignment.py</code>	<code>fit_rigid_svd</code> , <code>nearest_correspondences</code>
<code>task2_ransac.py</code>	<code>ransac_rigid</code>
<code>task3_odometry.py</code>	<code>icp</code> , <code>chain_relative_poses</code> , <code>relative_trajectory_error</code>
<code>task4_mapping.py</code>	<code>points_to_voxels</code> , <code>fuse_voxel_evidence</code>

4 Task 1: Rigid alignment and metrics [3.0 points]

Two scans of the same scene live in different sensor frames. Estimate the rigid motion that brings matched source points $\mathbf{p}_i$ near target points $\mathbf{q}_i$.

1A. Rigid alignment with SVD [1.8 points]

Implement `fit_rigid_svd(source, target)`:

$$\min_{\mathbf{R} \in SO(3), \mathbf{t}} \sum_i \|\mathbf{R}\mathbf{p}_i + \mathbf{t} - \mathbf{q}_i\|_2^2.$$

Both inputs are homogeneous $4 \times N$ point matrices; operate on their first three rows when computing centroids and covariance. Define $\mathbf{H} = \mathbf{P}_c \mathbf{Q}_c^\top$, where the centered XYZ point columns form $\mathbf{P}_c$ and $\mathbf{Q}_c$. With $\mathbf{H} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top$, use

$$\mathbf{D} = \text{diag}(1, 1, \det(\mathbf{V}\mathbf{U}^\top)), \quad \mathbf{R} = \mathbf{V}\mathbf{D}\mathbf{U}^\top, \quad \mathbf{t} = \bar{\mathbf{q}} - \mathbf{R}\bar{\mathbf{p}}.$$

The determinant correction prevents a reflection. Return one finite 4×4 transform. Checkpoint 1A writes `estimated_transform.txt`.

Before coding, check the direction of your result: after applying $\mathbf{T}_{\text{source} \rightarrow \text{target}}$, which frame should the transformed points live in? Also check why $\det(\mathbf{R}) = -1$ is not a valid 3D rotation even when its least-squares residual is small.

1B. Correspondences and metrics [1.2 points]

Implement `nearest_correspondences(source, target, max_distance)`. Build a `scipy.spatial.cKDTree` on the target, query one neighbor per source point, and reject a match only when its distance is greater than the threshold. Return aligned source indices, target indices, and Euclidean distances.

Checkpoint 1B writes `svd_alignment.png` and `svd_metrics.json`. Report translation error, rotation error, inlier RMSE, and inlier ratio. Explain why a small inlier RMSE is incomplete without the inlier ratio.

5 Task 2: Robust registration with RANSAC [2.5 points]

Known pair indices can still include wrong matches. RANSAC estimates motion from a minimal subset and measures how many other pairs agree.

Implement `ransac_rigid(source, target, threshold, max_trials, rng)`:

1. sample three distinct, non-collinear paired point columns;
2. estimate a transform with Task 1A;
3. score all pairs by Euclidean residual;
4. retain the largest consensus set; and
5. refit from all final inliers.

Use only the supplied random generator. Skip degenerate samples without crashing. Test both triples for degeneracy using cross-product norm below 10^{-8} square meters. Keep the first hypothesis when consensus sizes tie. If no hypothesis has three inliers, return identity and an all-false mask. After the single refit, recompute the returned Boolean mask using the final transform and the inclusive residual threshold. Checkpoints write `ransac_transform.txt`, `ransac_alignment.png`, and `ransac_metrics.json`. Compare all-pairs SVD and RANSAC, then explain what happens when the inlier threshold is much too small or much too large.

Before coding, ask why three paired points are the minimal useful sample, and why collinear triples cannot constrain every rotational degree of freedom.

6 Task 3: Real-data ICP and odometry [4.5 points]

One aligned pair gives one relative motion. Chaining many relative motions gives a trajectory, but every local error affects all later poses. You will first debug ICP on a manageable real pair, then use exactly the same function on the LiDAR sequence.

3A. ICP on the Open3D demo [1.8 points]

Implement `icp(source, target, ...)` with the defaults in the starter. Both point sets are homogeneous $4 \times N$ matrices. At each iteration:

1. transform the original source using the current estimate;
2. call your Task 1 nearest-neighbor function;
3. retain the lowest-residual `trim_fraction` matches;
4. fit an incremental transform with Task 1A and left-compose it; and
5. stop when the absolute RMSE change is below `tolerance` or the iteration limit is reached.

Return $\mathbf{T}_{\text{source} \rightarrow \text{target}}$, the RMSE history, and a transform history containing the initial identity followed by every update. If fewer than three accepted matches remain, stop and return the current transform and histories. Otherwise retain $\max(3, \lceil fK \rceil)$ of the K accepted matches, where f is `trim_fraction`. Measure RMSE after the update on that iteration's retained pairs without re-matching. Compare consecutive recorded RMSE values; do not test convergence before recording the first value. Do not call an off-the-shelf ICP implementation. The first checkpoint runs on the real Open3D fragments from a deliberately perturbed initialization and writes `icp_demo_alignment.png` plus `icp_iterations.mp4`. Both use Open3D surfels with estimated normals and one fixed, front-facing RGB-D camera; verify that the source converges toward the target rather than mistaking camera motion for alignment.

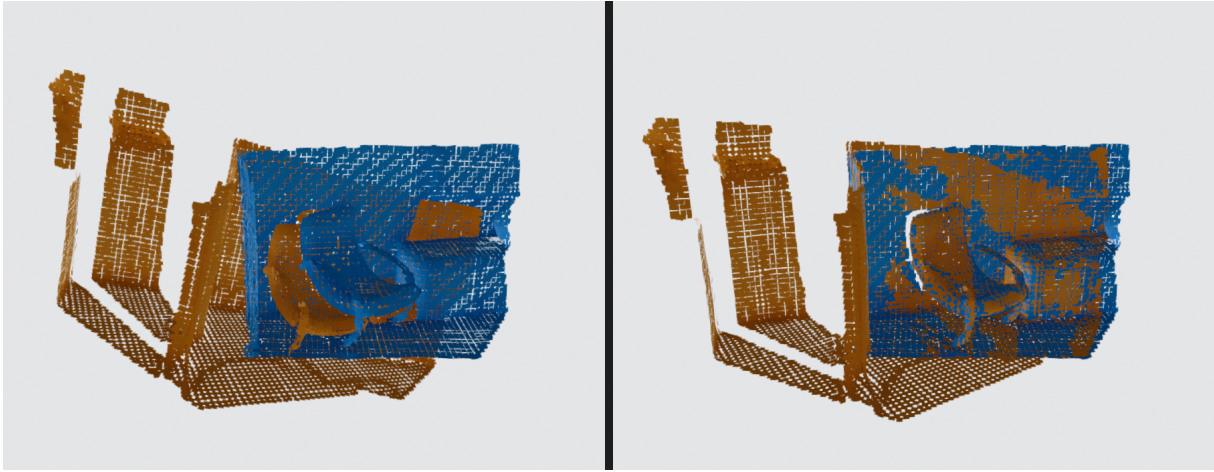


Figure 3: Checkpoint 3A: a real two-fragment registration problem before and after the handwritten ICP implementation, rendered from the same front-facing RGB-D viewpoint.

3B. Curved LiDAR odometry [1.2 points]

The pipeline applies the same `icp` function to all 1,023 adjacent LiDAR pairs. It uses the previous relative motion as the next initial guess; this is supplied code. Implement `chain_relative_poses(relative_transforms)`. Each supplied relative transform is $\mathbf{T}_{L_{t+1} \rightarrow L_t}$. Each chained pose is $\mathbf{T}_{L_t \rightarrow W}$. Initialize $\mathbf{T}_{L_0 \rightarrow W} = \mathbf{I}$ and compose

$$\mathbf{T}_{L_{t+1} \rightarrow W} = \mathbf{T}_{L_t \rightarrow W} \mathbf{T}_{L_{t+1} \rightarrow L_t}.$$

Do not add translations or Euler angles component-wise. Checkpoint 3B writes `icp_convergence.png`.

Before coding, follow one point from L_{t+1} through the two transforms in the equation. Its frame after each multiplication tells you the correct composition order. Consider also why the previous relative motion is a useful initial guess for the next pair, but not a guaranteed one.

3C. Trajectory metrics and visualization [1.5 points]

Implement `relative_trajectory_error(estimated_poses, ground_truth_poses, delta=1)`. For pairs separated by `delta`, use

$$\Delta \hat{\mathbf{T}}_i = \hat{\mathbf{T}}_{L_i \rightarrow W}^{-1} \hat{\mathbf{T}}_{L_{i+\delta} \rightarrow W}, \quad \Delta \mathbf{T}_i = \mathbf{T}_{L_i \rightarrow W}^{-1} \mathbf{T}_{L_{i+\delta} \rightarrow W}, \quad \mathbf{E}_i = \Delta \mathbf{T}_i^{-1} \Delta \hat{\mathbf{T}}_i.$$

Translation error is $\|\mathbf{E}_i[0:3, 3]\|_2$. Rotation error in degrees is computed from

$$\cos^{-1} \left(\text{clip} \frac{\text{tr}(\mathbf{R}_{E_i}) - 1}{2}, -1, 1 \right).$$

The final checkpoint writes `trajectory_comparison.png`, `bev_comparison.png`, `icp_odometry_online.mp4`, and `metrics.json`. The 1,024-frame online video keeps the current LiDAR at the center of a heading-up ± 50 m BEV. Its left panel rasterizes every point in the current stored sweep without an additional render-time downsample; its right panel contains the intensity map accumulated only through that frame and overlays the inferred ICP trajectory up to the current pose in cyan. Both static figures compare only GT and ICP. The trajectory axes use equal scale. The two BEV panels use the same physical extent and resolution; doubled or smeared structures expose drift. Report ATE, mean one-step translation RTE, and mean one-step rotation RTE. Identify where drift is visible in both the path and BEV. Here ATE means XY translation RMSE after setting the first pose to identity; no further trajectory alignment is performed.

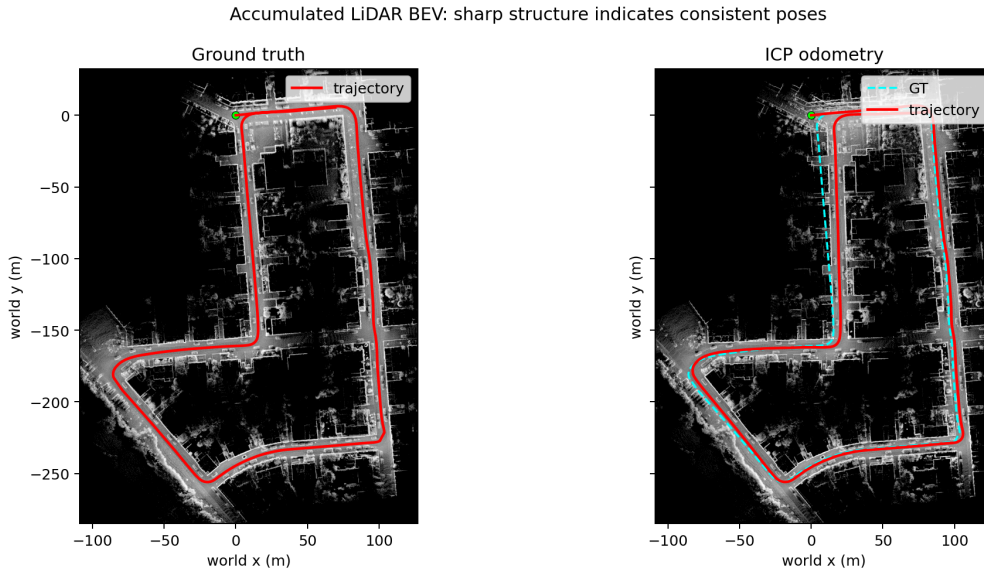


Figure 4: GT and ICP accumulated LiDAR maps at a shared metric scale.

7 Task 4: Semantic voxel mapping [4.0 points]

Now combine all 1,024 semantically labeled scans into one sparse 3D occupancy map. This task uses the supplied GT poses so that it isolates voxel conversion and semantic fusion from odometry drift.

Before coding: can the grid be dense? Suppose 0.25 m cubical voxels cover a $500 \text{ m} \times 500 \text{ m} \times 20 \text{ m}$ region and each voxel stores 19 float32 class scores. In your report, estimate the dense tensor shape, total number of voxels, and memory required for the scores alone. Do not allocate this tensor. Hint: for side length L and voxel size s , use $N = \lceil L/s \rceil$ cells; float32 uses four bytes. How does your estimate motivate a sparse representation?

4A. Point-to-voxel conversion [0.6 points]

For world point $\mathbf{p}_W$, voxel size s , and origin $\mathbf{o}$, implement

$$\mathbf{v} = \left\lfloor \frac{\mathbf{p}_W - \mathbf{o}}{s} \right\rfloor$$

in `points_to_voxels(points, origin, voxel_size)`. Return signed integer `torch.long` indices and do not clip them. The fusion function handles bounds. The inputs are a homogeneous $[4, N]$ PyTorch point

tensor and a [3] origin tensor. Divide the first three rows by the homogeneous row, then produce $[N, 3]$ voxel indices. Why does `floor` give the correct cell for a point just below the origin while rounding does not?

4B. Sparse semantic evidence fusion [2.4 points]

Implement `fuse_voxel_evidence(voxel_indices, raw_logits, grid_shape)`. Reject out-of-bounds indices, compute stable log-softmax scores, merge repeated coordinates, and accumulate evidence:

$$o_v = \mathbf{1}[n_v > 0], \quad \mathbf{s}_v = \sum_{i \in v} \log \text{softmax}(\ell_i).$$

Return PyTorch tensors: lexicographically sorted coordinates $[M, 3]$, binary occupancy $[M]$, summed log evidence $[M, C]$, and observation counts $[M]$. Never overwrite a voxel with its last observation. Use PyTorch tensor operations such as `torch.unique`, `torch.log_softmax`, and `index_add_` (or equivalent); do not loop over points. Accumulate the log evidence in float64 so exact class-count ties remain deterministic. The supplied logits are int8: convert them to float64 *before* calling log-softmax.

These coordinate/value tensors use the same sparse idea as COO storage: keep only occupied coordinates and their features. You do not need to construct a `torch.sparse_coo_tensor`; the supplied renderer consumes the returned coordinate and value tensors. For context, see <https://docs.pytorch.org/docs/stable/sparse.html>. Before implementation, consider why summing log evidence lets repeated observations reinforce one another, while keeping only the last observation discards useful information.

4C. Full-scene 3D result [1.0 point]

The supplied pipeline transforms every one of the 1,024 frames. After the range and height filters, it retains at most 2,500 evenly spaced scan-order samples per frame to keep memory practical. It uses 0.25 m voxels, evaluates the fused labels against per-voxel GT majority, and writes:

- `semantic_metrics.json` and `voxel_statistics.json`;
- `semantic_occupancy.ply`;
- `semantic_occupancy.png`, an Open3D shoulder view; and
- `semantic_occupancy.mp4`, a 96-frame turntable video; and
- `semantic_lidar_follow.mp4`, all 1,024 frames with the current LiDAR scan and the full semantic map shown side by side from the identical virtual camera, 3 m above the LiDAR and looking forward/down.

The left panel renders every point in the stored sweep without the map's filter or 2,500-point cap, colored by range with the `inferno` colormap.

Each occupied voxel is an Open3D cube primitive. Its physical side length is derived from the voxel size; the renderer never stretches one axis to fit the window. Report voxel accuracy, mean IoU, per-class IoU for present classes, occupied voxel count, observation-count statistics, and mean normalized entropy. Explain one region where labels from different frames compete and one limitation of treating all observations as equally reliable. The supplied entropy uses softmax of *mean* log evidence, describing class agreement rather than a posterior whose confidence grows with repeated observations. GT majority is generated independently of your voxel coordinates; the `voxel_coordinates_correct` field must be true before semantic metrics are reported. High semantic accuracy alone does not validate geometry. Relate the 0.25 m voxel size to both geometric detail and memory.

8 Optional bonus: verified loop closure [+1.0 point]

Gen-AI leverage allowed

The vehicle revisits a similar place with the opposite heading. Use a geometrically verified loop closure to correct accumulated odometry drift.

Using the same `sequence00.npz`, find at least one pair separated by 400 or more original archive indices that revisits the same place. Candidate selection must use geometry, not GT poses, tolerate opposite headings, and reject geometrically false matches. Verify the pair with RANSAC, pass the accepted edge to the provided `pose_graph.py` optimizer, and compare trajectory and pose-graph residuals before and after optimization. You may use the Task 3 odometry poses, but reference poses may be loaded only after the pair has been retrieved and geometrically verified, for evaluation. The optimizer accepts planar poses $[K, 3]$ as (x, y, θ) and edges $[E, 6]$ as $(i, j, dx, dy, d\theta, w)$. Translation is in meters, yaw in radians, and $w > 0$. Each edge maps frame j into frame i : $\mathbf{T}_{j \rightarrow i} = \mathbf{T}_{i \rightarrow W}^{-1} \mathbf{T}_{j \rightarrow W}$. Node zero is fixed; all nodes must be connected to it. Use a compact keyframe graph to limit dense-solver cost. Run `pythonpose_graph.py` for a three-node example.

No bonus starter, retrieval method, or alignment helper is provided. Submit your own directly runnable `bonus_loop_closure.py`. It must save the chosen pair, registration statistics, trajectory comparison, and metrics.

9 Report and submission [1.0 point]

Use `report_template.tex`; 4–6 pages is recommended. Every number and figure must be produced by your submitted scripts. Include:

- SVD and RANSAC metrics plus both alignment figures;
- the real Open3D ICP before/after figure, both Task 3 video filenames, convergence plot, GT/ICP trajectory, GT/ICP BEV, ATE, RTE, and a short observation from the current-sweep/online-map comparison;
- the semantic cube-map PNG, both video filenames, voxel metrics/statistics, dense-memory estimate, and conflict analysis; and
- one concise failure analysis tied to a geometric or fusion assumption.

Submit the revised `task1_alignment.py`, `task2_ransac.py`, and `task3_odometry.py`. Also submit `task4_mapping.py` and one PDF report. If attempted, also submit `bonus_loop_closure.py`. Do not submit data, helpers, checkpoints, or generated maps. Run each required script once more from the directory containing the task scripts and confirm that saved results match the report.

10 Integrity and generative-AI policy

This is an individual assignment. Required functions, analysis, figures, and report writing must be your own. Generative AI tools such as GitHub Copilot, Codex, ChatGPT, Claude, and similar systems may not be used for required tasks. Generative AI may be used only in the optional bonus marked **Gen-AI leverage allowed**; disclose the tool, purpose, and affected files.

Course staff may ask you to explain your implementation, discuss version history, answer a submission-specific question, or make a small related change. Report, code, outputs, and numerical results must be mutually consistent. A document marker or automated detector is only a review signal and is never, by itself, evidence of misconduct.

References. S. Umeyama, “Least-Squares Estimation of Transformation Parameters Between Two Point Patterns,” 1991. M. A. Fischler and R. C. Bolles, “Random Sample Consensus,” 1981. P. J. Besl and N. D. McKay, “A Method for Registration of 3-D Shapes,” 1992. J. Behley et al., “SemanticKITTI,” 2019. Open3D ICP tutorial, https://www.open3d.org/docs/latest/tutorial/pipelines/icp_registration.html. PyTorch sparse tensors, <https://docs.pytorch.org/docs/stable/sparse.html>.

Data: <https://semantic-kitti.org/dataset.html> and <https://www.cvlibs.net/datasets/kitti/>.